

11. Write a program to implement following operations on the circular linked list. (a) Insert a node at the end of the linked list. (b) Insert a node before specified position. (c) Delete a first node of the linked list. (d) Delete a node after specified position.

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head=NULL;

/* Insert at End */
void insertEnd(int val)
{
    struct node *newnode,*temp;

    newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=val;

    if(head==NULL)
    {
        head=newnode;
        newnode->next=head;
    }
    else
    {
        temp=head;
        while(temp->next!=head)
        {
            temp=temp->next;
        }

        temp->next=newnode;
        newnode->next=head;
    }

    printf("Node inserted at end\n");
}

/* Insert Before Position */
void insertBeforePos(int val,int pos)
{
    struct node *newnode,*temp;
```

```

int i;

newnode=(struct node*)malloc(sizeof(struct node));
newnode->data=val;

if(pos==1)
{
    temp=head;
    while(temp->next!=head)
        temp=temp->next;

    newnode->next=head;
    temp->next=newnode;
    head=newnode;
}
else
{
    temp=head;

    for(i=1;i<pos-1;i++)
        temp=temp->next;

    newnode->next=temp->next;
    temp->next=newnode;
}

printf("Node inserted before position\n");
}

/* Delete First Node */
void deleteFirst()
{
    struct node *temp,*last;

    if(head==NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp=head;

    if(head->next==head)
    {
        head=NULL;
    }
    else
    {
        last=head;

```

```

        while(last->next!=head)
            last=last->next;

        head=head->next;
        last->next=head;
    }

    free(temp);
    printf("First node deleted\n");
}

/* Delete Node After Position */
void deleteAfterPos(int pos)
{
    struct node *temp,*del;
    int i;

    temp=head;

    for(i=1;i<pos;i++)
        temp=temp->next;

    del=temp->next;

    temp->next=del->next;

    free(del);

    printf("Node deleted after position\n");
}

/* Display List */
void display()
{
    struct node *temp;

    if(head==NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp=head;

    do
    {
        printf("%d -> ",temp->data);
        temp=temp->next;
    }

```

```

while(temp!=head);

printf("HEAD\n");
}

/* Main Function */
int main()
{
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);

    display();

    insertBeforePos(15,2);
    display();

    deleteFirst();
    display();

    deleteAfterPos(2);
    display();

    return 0;
}

```

Output

```

Node inserted at end
Node inserted at end
Node inserted at end
10 -> 20 -> 30 -> HEAD
Node inserted before position
10 -> 15 -> 20 -> 30 -> HEAD
First node deleted
15 -> 20 -> 30 -> HEAD
Node deleted after position
15 -> 20 -> HEAD

```